

Instruções de Comando DML



Conteudista: Prof. Me. Alexander Gobbato Albuquerque

Revisão Textual: Prof.^a Esp. Vera Lúdia de Sá Cicarone

Objetivo da Unidade:

- Compreender comandos para a manipulação de dados, comando *DDL* (*Data Definition Language*).



Material Teórico



Material Complementar



Referências



Material Teórico

Introdução

Com o conhecimento adquirido nos comandos *DML (Data Manipulation Language)*, aprenderemos, agora, como inserir, alterar e deletar informações.

Além de trabalharmos com os comandos acima, também veremos como podemos exibir as informações armazenadas.

Então está na hora de aprendermos os comandos *DDL (Data Definition Language)*.

Inserir linhas em tabelas:

- INSERT;
- UPDATE;
- DELETE;
- RENAME;
- CREATE TABLE ... AS SELECT.

Recuperar Dados de uma Tabela:

- `SELECT`;
- `Distinct`;
- Expressões Aritméticas;
- Cláusula *Where*;
- Alias.

Pré-requisito para este material:

Logar-se com o usuário HR e Selecionar *SQL Commands*.

Site

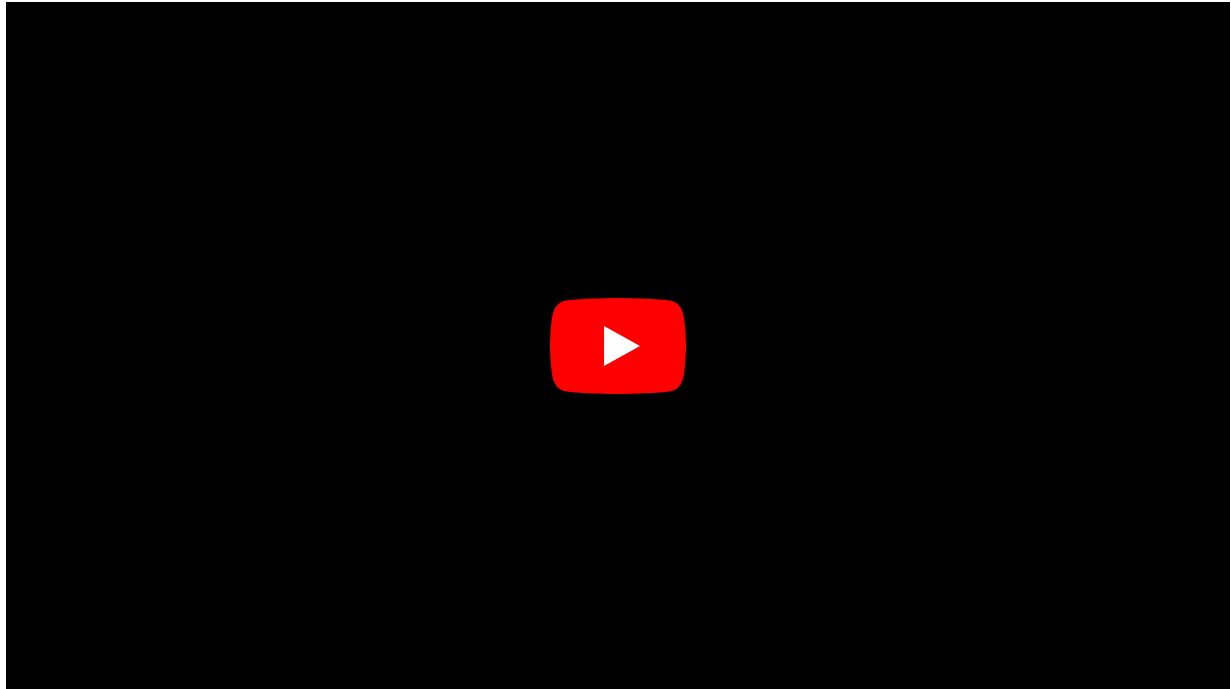
Crie a sua Conta *Oracle*

Clique no botão para conferir o conteúdo.

ACESSE

Vídeo

Acesso ao Oracle Live SQL



ORACLE® Database Express Edition

Database Login

Enter your database username and password.

Username

Password

[Click here to learn how to get started](#)

Links

- o [License Agreement](#)
- o [Documentation](#)
- o [Forum Registration](#)
- o [Discussion Forum](#)
- o [Product Page](#)

Figura 1

Fonte: Reprodução

#ParaTodosVerem: a imagem mostra uma tela de login para acessar um banco de dados *Oracle Express Edition*. Na parte superior, há o logotipo do *Oracle Database Express Edition* e um menu verde. Abaixo, há um campo para inserir o

nome de usuário (já preenchido com "HR") e outro para a senha. Ao lado direito, há uma lista com links para o contrato de licença, documentação, registro no fórum, fórum de discussão e página do produto. Na parte inferior, há um botão "Login" e um link para obter ajuda sobre como começar. Fim da descrição.

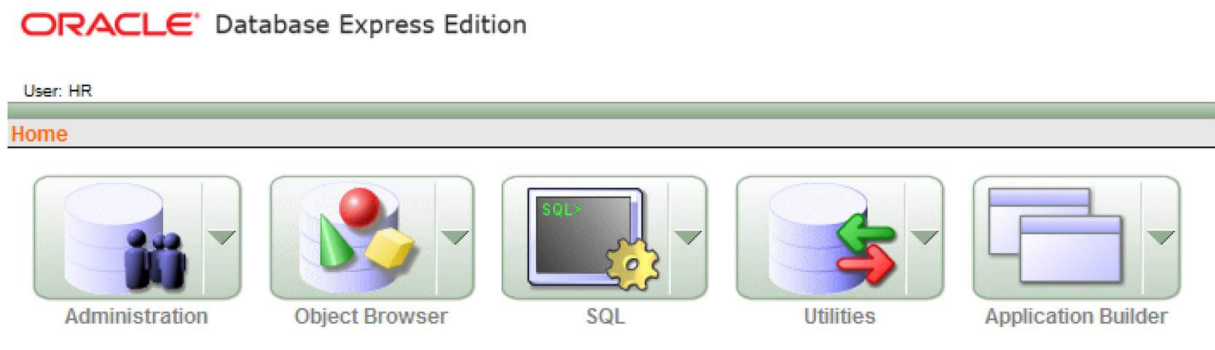


Figura 2

Fonte: Reprodução

#ParaTodosVerem: a imagem mostra a tela inicial de um programa chamado *Oracle Database Express Edition*. Na parte superior, aparece o nome do programa e o nome do usuário logado, que é "HR". Abaixo, há um menu com as seguintes opções: *Home*, *Administration*, *Object Browser*, *SQL*, *Utilities* e *Application Builder*. Cada opção é representada por um ícone diferente. Fim da descrição.

```
CREATE TABLE pais(  
  cd_pais number (2),  
  nm_pais VARCHAR (45),  
  ds_nacionalidade VARCHAR (35)  
)
```

Inserindo registros nas tabelas.

Sintaxe:

```
INSERT INTO nome_da_tabela  
[(nome_da_coluna[,nome_da_coluna])]  
VALUES  
(valor_da_coluna[,valor_da_coluna])
```

Valores alfanuméricos e datas deverão estar entre aspas simples ('...'):

- **Ex. 1:**

```
INSERT INTO pais (cd_pais, nm_pais) VALUES (1,'Brasil');
```

- **Ex. 2:**

```
INSERT INTO pais (cd_pais, nm_pais, ds_nacionalidade)VALUES (2,  
'Argentina', null);
```

- **Ex. 3:**

```
INSERT INTO pais VALUES (4, 'Chile', 'Chilena');
```

- **Ex. 4:**

```
INSERT INTO pais (nm_pais, ds_nacionalidade, cd_pais) VALUES ('EUA',  
'Americana' , 3);
```

Para incluir dados em uma tabela, pode-se, ou não, incluir valores em todas as suas colunas (caso não sejam obrigatórias).

A relação de colunas da tabela poderá ser omitida quando forem inclusos valores para todas as colunas dessa tabela.

Update

Modificar a(s) linha(s) existentes com o comando *UPDATE*.

```
UPDATE Tabela  
SET COLUNA = Valor  
WHERE Condição
```

Exemplos:

```
Update Paisset Nm_pais='Portugal',Ds_nacionalidade='Portug'  
WhereCd_pais = 3;
```

```
Update Paisset Ds_nacionalidade='Portuguesa'  
WhereCd_pais = 3;
```

Delete

Remover a(s) linha(s) existente(s) com o comando *DELETE*.

```
DELETE FROM Tabela  
WHERE Condição
```

Importante!

- Verifique os nomes das colunas com o comando *DESCRIBE*;
- Confirme a operação de atualização pela visualização das linhas a serem atualizadas com o comando *SELECT*;
- Nunca omita a cláusula *WHERE*. No caso da omissão, todos os registros da tabela serão eliminados.

```
DELETE FROM Pais WHERE Cd_pais = 3;
```

Rename

Este comando é utilizado nos casos de alteração de nome das tabelas.


```
RENAME nome_antigo_tabela TO nome_novo_tabela
```

Exemplo:

```
RENAME Pais TO Tb_Pais
```

Create Table as Select

Antes de começarmos este exemplo, em seu *Live Oracle* popule o mesmo.

Selecione o menu *Code Library* você tem opções de *scripts* prontos para serem baixados e utilizados em seu *Live Oracle*.

Este comando cria uma tabela (estrutura e dados) com base em um comando *SELECT*.

```
create table espelhoteste as select * from employees
```

Results	Explain	Describe	Saved SQL	History	Results	Explain	Describe	Saved SQL	History
EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER
100	Steven	King	SKING	515.123.4567	100	Steven	King	SKING	515.123.4567
101	Neena	Kochhar	NKOCHHAR	515.123.4568	101	Neena	Kochhar	NKOCHHAR	515.123.4568
102	Lex	De Haan	LDEHAAN	515.123.4569	102	Lex	De Haan	LDEHAAN	515.123.4569
103	Alexander	Hunold	AHUNOLD	590.423.4567	103	Alexander	Hunold	AHUNOLD	590.423.4567
104	Bruce	Ernst	BERNST	590.423.4568	104	Bruce	Ernst	BERNST	590.423.4568
105	David	Austin	DAUSTIN	590.423.4569	105	David	Austin	DAUSTIN	590.423.4569
106	Valli	Pataballa	VPATABAL	590.423.4560	106	Valli	Pataballa	VPATABAL	590.423.4560
107	Diana	Lorentz	DLORENTZ	590.423.5567	107	Diana	Lorentz	DLORENTZ	590.423.5567
108	Nancy	Greenberg	NGREENBE	515.124.4569	108	Nancy	Greenberg	NGREENBE	515.124.4569
109	Daniel	Faviet	DFAVIET	515.124.4169	109	Daniel	Faviet	DFAVIET	515.124.4169
More than 10 rows available. Increase rows selector to view more rows.					More than 10 rows available. Increase rows selector to view more rows.				

Figura 3

Fonte: Reprodução

#ParaTodosVerem: a imagem mostra duas tabelas com informações sobre funcionários. Cada tabela possui as seguintes colunas: ID do funcionário, primeiro nome, último nome, *e-mail* e número de telefone. As tabelas apresentam os mesmos dados, mas em formatos diferentes. As tabelas contêm mais de 10 linhas de dados, indicando que há mais informações sobre outros funcionários que não estão sendo mostradas na imagem. Fim da descrição.

A Figura anterior mostra os dados que foram copiados da tabela *employees* para a tabela *espelhoteste*.

Select

O comando *SELECT* permite a seleção e a manipulação para visualização das informações armazenadas no banco de dados.

O comando *SELECT* permite selecionar:

(Para selecionar essas tabelas, é preciso logar-se com o usuário HR no *Oracle*)

- Todas as colunas:

```
SELECT * FROM departments;
```

- **Colunas específicas:**

```
SELECT FIRST_NAME, EMAIL, HIRE_DATE FROM employees;
```

- **Expressões Aritméticas:**

```
SELECT FIRST_NAME, EMAIL , salary, salary*12 FROM employees;
```

- **Colunas com Apelidos (alias):**

```
SELECT FIRST_NAME, EMAIL, salary "Salário", salary*12 "SalarioAnual" FROM employees;
```

- **Colunas concatenadas:**

```
SELECT FIRST_NAME, EMAIL, 'Salário: ' || salary "Salário", salary*12 "Salário*12"  
FROM employees;
```

Linhas Duplicadas

A exibição default de consultas são todas as linhas, incluindo as linhas duplicadas.

Para eliminar valores duplicados no resultado, incluímos o *DISTINCT* qualificador no comando *SELECT*.

```
SELECT distinct department_id FROM employees  
ORDER BY department_id;
```

Cláusula *WHERE*

A Cláusula *WHERE* indica condição para um *SELECT*, o qual pode ter os seguintes operadores:

Tabela 1

Operador	Descrição
----------	-----------

=	Igual
<>	Diferente
>	Maior
<	Menor
>=	Maior ou igual
<=	Menor ou igual

Alfanuméricos e Datas na cláusula *WHERE* devem estar entre aspas simples.

Operadores SQL

Existem quatro operadores SQL os quais opera, com todos tipos de dados:

Tabela 2

Operador	Significado
<i>Between... and...</i>	Entre dois valores (inclusive).
<i>In (lista)</i>	Comparar uma lista de valores.

<i>Like</i>	Compara um parâmetro alfanumérico.
<i>Is Null</i>	É um valor nulo.

O Operador *BETWEEN*

Compara uma faixa de valores, inclusive o menor e o maior valor.

Suponhamos que nós queiramos ver aqueles empregados cujos salários estejam entre 1000 e 2000:

```
SELECT First_name, Last_name, Salary
FROM Employees
WHERE Salary BETWEEN 10000 AND 20000;
```

Tabela 3

FIRST_NAME	LAST_NAME	SALARY
Neena	Kochhar	17000
Lex	De Haan	17000
Nancy	Greenberg	12008

Den	Raphaely	11000
John	Russell	14000
Karen	Partners	13500
Alberto	Errazuriz	12000
Gerald	Cambrault	11000
Eleni	Zlotkey	10500
Peter	Tucker	10000
Janette	King	10000
Clara	Vishney	10500
Lisa	Ozer	11500
Harrison	Bloom	10000
Ellen	Abel	11000
Michael	Hartstein	13000
Hermann	Baer	10000

Shelley	Higgins	12008
18 rows selected		

Note que inclusive os valores especificados estão no resultado, e o menor precisa ser especificado primeiro.

O Operador *IN*

Compara os valores especificados dentro de uma lista.

Para encontrar empregados que tenham um dos três números de Manager, utilize o seguinte comando:

```
SELECT First_name, Last_name, Salary, Manager_id
FROM Employees
WHERE Manager_id IN (101, 145, 149);
```

Tabela 4

FIRST_NAME	LAST_NAME	SALARY	MANAGER_ID
Nancy	Greenberg	12008	101
Peter	Tucker	10000	145

David	Bernstein	9500	145
Peter	Hall	9000	145
Christopher	Olsen	8000	145
Nanette	Cambrault	7500	145
Ellen	Abel	11000	149
Alyssa	Hutton	8800	149
Jonathon	Taylor	8600	149
Jack	Livingston	8400	149
Kimberely	Grant	7000	149
Charles	Johnson	6200	149
Jennifer	Whalen	4400	101
Susan	Mavris	6500	101
Hermann	Baer	10000	101
Shelley	Higgins	12008	101

17 rows selected

Se alfanuméricos ou datas forem usados na lista, precisam ser colocados entre aspas simples ('...').

O Operador *LIKE*

Algumas vezes você precisa procurar valores que não conhece exatamente. Usando o operador *LIKE*, é possível selecionar linhas combinando parâmetros alfanuméricos. O caractere % é utilizado como coringa nas pesquisas. Veja exemplo a seguir:

Tabela 5

Sintaxe	Descrição
<i>Like</i> '%cadeia'	Localiza valores que terminem com a cadeia de caracteres.
<i>Like</i> 'cadeia%'	Localiza valores que comecem com a cadeia de caracteres.
<i>Like</i> '%cadeia%'	Localiza valores que tenham a cadeia de caracteres em qualquer parte do campo.

Para listar todos os empregados que tenham o nome que comecem com a letra S, faça:

```
SELECT First_name, Last_name, Salary
FROM Employees
WHERE First_name like 'S%';
```

O Operador *IS NULL*

Verifica quais campos estão com valores nulos.

Unicamente encontrar todos os empregados que não tenham gerente, você testará um valor nulo:

```
SELECT First_name, Last_name, Salary, Manager_id
FROM Employees
WHERE Manager_id IS Null;
```

Expressões Negativas

Podemos também negar todos os operadores estudados:

Tabela 6

Sintaxe	Descrição
<i>NOT Between</i>	Tudo que estiver fora da faixa.

<i>NOT In</i>	Tudo que não estiver na lista.
<i>NOT Like</i>	Tudo que não contiver a linha de caracteres.
<i>IS NOT Null</i>	Tudo que não for nulo.

```
SELECT First_name, Last_name, Salary
FROM Employees
WHERE Salary NOT BETWEEN 10000 AND 20000;
SELECT First_name, Last_name, Salary, Manager_id
FROM Employees
WHERE Manager_id NOT IN (101, 145, 149);
```

```
SELECT First_name, Last_name, Salary
FROM Employees
WHERE First_name NOT Like 'S%';
```

```
SELECT First_name, Last_name, Salary, Manager_id
FROM Employees
WHERE Manager_id IS NOT Null;
```

Pesquisando Dados com Múltiplas Condições

- Os operadores AND e OR devem ser usados para fazer composições de expressões lógicas.
- O predicado AND esperará que ambas as condições sejam verdadeiras;
- O predicado OR esperará que uma das condições seja verdadeira.

Nos dois exemplos seguintes, as condições são as mesmas, mas o predicado é diferente.

Veja como o resultado é dramaticamente alterado.

Para encontrar todos os escriturários do departamento de estoque que ganhem entre 2000 e 3000.

```
SELECT First_name, Last_name, Salary, Job_ID
FROM Employees
WHERE Salary BETWEEN 2000 AND 3000
AND Job_id = 'ST_CLERK';
```

Tabela 7

FIRST_NAME	LAST_NAME	SALARY	JOB_ID
Irene	Mikkilineni	2700	ST_CLERK

James	Landry	2400	ST_CLERK
Steven	Markle	2200	ST_CLERK
Mozhe	Atkison	2800	ST_CLERK
James	Marlow	2500	ST_CLERK
TJ	Olson	2100	ST_CLERK
Michael	Rogers	2900	ST_CLERK
Ki	Gee	2400	ST_CLERK
Hazel	Phitanker	2200	ST_CLERK
John	Seo	2700	ST_CLERK
Joshua	Patel	2500	ST_CLERK
Randall	Matos	2600	ST_CLERK
Peter	Vargas	2500	ST_CLERK
13 rows selected			

Para encontrar todos os escriturários do departamento de estoque e os funcionários que ganhem entre 2000 e 3000.

```
SELECT First_name, Last_name, Salary
FROM Employees
WHERE Salary BETWEEN 2000 AND 3000
OR Job_id = 'ST_CLERK';
```

Perceba que a seleção foi bem maior, visto que existem funcionários que ganham entre 2000 e 3000 e não são escriturários do departamento de estoque.

Tabela 8

FIRST_NAME	LAST_NAME	SALARY
Shelli	Baida	2900
Sigal	Tobias	2800
Guy	Himuro	2600
karen	Colmenares	2500
Julia	nayer	2300
Irene	Mikkilineni	2700

James	Landry	2400
Steven	Markle	2200
Laura	Bissot	3300
Mozhe	Atkinson	2800
James	Marlow	2500
TJ	Olson	2100
Jason	Mallin	3300
Michael	Rogers	2900
Ki	Gee	2400
Hazel	Philtanker	2200
Renske	Ladwig	3600
Stephen	Stiles	3200
John	Seo	2700
Joshua	Patel	2500

Trenna	Rajs	3500
Curtis	Davies	3100
Randall	Matos	2600
Peter	Vargas	2500
Martha	Sullivan	2500
Girard	Geoni	2800
Anthony	Cabrio	3000
Timothy	Gates	2900
Randall	Perkins	2500
Vance	Jones	2800
Kevin	Feeney	3000
Donald	OConnell	2600
Douglas	Grant	2600
33 rows selected		

Você pode combinar AND e OR na mesma expressão lógica.

Quando AND e OR aparecerem na mesma cláusula WHERE, todos os ANDs serão feitos primeiro e, posteriormente, todos os “ORs” serão feitos.

```
SELECT First_name, Last_name, Salary, Job_ID
FROM Employees
WHERE Salary > 8000 and
Job_id = 'ST_CLERK' OR
Job_id = 'ST_MAN'
```

Se você quiser selecionar todos os Escriturários e Gerentes que ganhem acima de 8000, deverá fazer:

```
SELECT First_name, Last_name, Salary, Job_ID
FROM Employees
WHERE Salary > 8000 and
(Job_id = 'ST_CLERK' OR
Job_id= 'ST_MAN')
```

Os parênteses especificam a ordem na qual os operadores devem ser avaliados (prioridade).

No segundo exemplo, o operador OR é avaliado antes do AND.

Sempre que você tiver dúvida sobre qual dos dois operadores será feito primeiro, quando a expressão é avaliada, use sempre parênteses para definir a prioridade das expressões.

A Cláusula *ORDER BY*

Normalmente a ordem das linhas retornadas de uma pesquisa é indefinida. A cláusula *ORDER BY* pode ser usada para ordenar as linhas.

Se usado, o *ORDER BY* deve ser sempre a última cláusula da declaração *SELECT*. Para ordenar essa consulta pelo campo *Last_Name*, faça:

```
SELECT First_name, Last_name, Salary
FROM Employees
WHERE Salary BETWEEN 10000 AND 20000
ORDER BY Last_name;
```

Padrão da Ordenação dos Dados

O padrão da ordem de ordenação é ascendente:

- Valores numéricos infinitos primeiro;
- Valores de data primeiro;
- Valores alfanuméricos.

Invertendo o Padrão de Ordenação

Para inverter essa ordem, acrescente o comando DESC (Decrescente) depois do nome das colunas da cláusula ORDER BY.

Para inverter a ordem da coluna Last_name, faça:

```
SELECT First_name, Last_name, Salary
FROM Employees
WHERE Salary BETWEEN 10000 AND 20000
ORDER BY Last_name DESC;
```

Ordenação por Várias Colunas

É possível utilizar mais de uma coluna na cláusula ORDER BY.

O limite de colunas é o número de colunas da tabela. Na cláusula ORDER BY, especificam as colunas que serão ordenadas, separando-as por vírgula. Se algumas ou todas forem invertidas, especifique DESC depois de cada uma das colunas.

Para ordenar por duas colunas e mostrar ordem inversa do salário e ordem crescente do sobrenome faça:

```
SELECT First_name, Last_name, Salary
FROM Employees
WHERE Salary BETWEEN 10000 AND 20000
ORDER BY Salary Desc, Last_name;
```

Para ordenar por uma coluna, ela não precisa, necessariamente, estar declarada no `SELECT`.

A cláusula `ORDER BY` é usada, na pesquisa, quando você quer mostrar as linhas em uma ordem específica. Sem a cláusula `ORDER BY`, as linhas são retornadas, na ordem conveniente, para o ORACLE.

Esse comando não altera a ordem dos dados que estão armazenados no Banco de Dados.



Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Vídeo

MYSQL – Aprenda na Prática SQL, Criar Banco, Tabelas, Consultas, Inserir Registros





Referências

COSTA, R. L. de C. **SQL: guia prático**. 2. ed. Rio de Janeiro: Brasport, 2006.

FANDERUFF, D. **Dominando o Oracle 9i: Modelagem e desenvolvimento**. São Paulo: Pearson Education do Brasil, 2003.

MORELLI, E. M. T. **Oracle 9i Fundamental: Sql, Pl/SQL e Administração**. 1996. São Paulo: Érica, 2002.

SILBERSCHATZ, A. **Sistema de bancos de dados**. São Paulo: *Pearson Education* do Brasil, 2004.